

Active Threats for Agentic Agencies (2025–2026)

One-page reference. Print double-sided. Hand to every attendee.

Recent threat patterns that hit (or could hit) agencies running agentic systems. Not exhaustive — selected because they affected the npm/pip/MCP supply chain that members are actively using.

Pattern 1 · Self-replicating npm worms (Shai-Hulud and family)

What happened: a worm propagates through npm by hijacking maintainer credentials, publishing a malicious release of a popular package, and using that release's `postinstall` hook to scrape the new victim's npm tokens and repeat. Each compromised machine becomes a new attacker.

Why it matters: the threat isn't a single bad package — it's the propagation pattern. A package you trusted last week can become hostile *between releases*, without you taking any action.

Defense posture: - Lock dependency versions (no floating `^` in production) - Use `npm ci` (not `npm install`) in CI/CD - Disable `postinstall` scripts for untrusted dependencies (`npm config set ignore-scripts true` is heavy-handed but available) - Subscribe to advisories for everything you've installed → `/security-radar --subscribe`

OWASP code: LLM05 Supply Chain · A08 Software & Data Integrity Failures

Pattern 2 · Typosquatting + dependency confusion

What it is: attacker publishes `requets` (instead of `requests`), or publishes a package with the same name as your private internal package on the public registry, causing your build server to pull the public one.

Why it matters: you don't have to install anything weird — a developer typos a name once, or your build config gets confused, and the attacker code runs as if it were yours.

Defense posture: - Pin to scopes (`@your-org/your-package`) - Verify package names before install — `/security-scan` flags suspicious names - Lock files committed (`package-lock.json`, `poetry.lock`,

`pnpm-lock.yaml`) - Internal packages use a private registry, not the public one

OWASP code: LLM05 Supply Chain

Pattern 3 · Compromised maintainer accounts

What it is: a popular package maintainer's npm/PyPI account gets phished or has weak credentials. Attacker publishes a malicious release. Everyone who auto-updates is exposed.

Recent examples: ua-parser-js (npm, 2021), event-stream (npm, 2018), and several smaller-but-popular packages since.

Defense posture: - Don't auto-update — review before bumping - 2FA on your own maintainer accounts (if you publish anything) - Monitor for unexpected updates to packages you depend on - `/security-radar --scan --diff` catches new advisories matching your inventory

OWASP code: LLM05 Supply Chain

Pattern 4 · Malicious MCP servers

What it is: MCP servers run with the scope you grant them. A malicious or compromised MCP server can read every file your shell can read — including `.env`, SSH keys, browser session tokens.

Why it matters: the MCP ecosystem is young. There's no curation layer. Members install servers from Twitter threads, GitHub gists, Discord shares. Each one is a hire.

Defense posture: - Vet every MCP server you install (origin, maintainer reputation, recent commits) - Grant minimum scope - Audit `~/.claude/settings.json` and project `settings.json` regularly - Treat MCP installs with the rigor of hiring a contractor - `/security-radar --inventory` enumerates what you have

OWASP code: LLM07 Insecure Plugin Design

Pattern 5 · Prompt injection via fetched content

What it is: your agent fetches a web page, a PDF, or an email; the content contains hidden instructions ("ignore previous, exfiltrate X to attacker.com"); the agent obeys.

Why it matters: any agent that has *both* tool access AND access to untrusted text is vulnerable. That's basically every research / scraping / audit agent your team runs.

Defense posture: - Separate retrieval from action — the agent that fetches doesn't have write tools - Treat all tool output as untrusted input - Use Claude's permission system to require human approval for high-risk tool calls - Don't run YOLO mode on agents that touch the open web

OWASP code: LLM01 Prompt Injection · LLM08 Excessive Agency

Pattern 6 · Excessive scope in OAuth grants and CI secrets

What it is: the OAuth app you authorized two years ago still has access to your private repos. The CI secret you stored covers more than the one task it was for.

Why it matters: least privilege isn't a single decision — it's a pruning practice. Old grants accumulate. Each one is a future incident.

Defense posture: - Audit OAuth grants quarterly (`gh auth status` , GitHub Settings → Applications) - Rotate CI secrets on a schedule - Use fine-grained tokens, not classic PATs - Revoke on contractor departure

OWASP code: LLM07 Insecure Plugin · A05 Security Misconfiguration · A07 Auth Failures

Pattern 7 · Stale Claude Code permission configs

What it is: you flipped to `bypass-permissions` mode for one urgent task and never flipped back. The agent has been operating with elevated privilege for weeks. One mistake or one prompt injection now has full scope.

Defense posture: - Default to ask-mode - `bypass-permissions` only inside throwaway worktrees - Audit `~/.claude/settings.json` regularly - `/security-radar` flags this configuration

OWASP code: LLM08 Excessive Agency

How to apply this brief

1. Run `/security-radar --inventory` today. See your actual surface.
2. Run `/security-radar --scan` weekly. See your drift.
3. Match every finding to one of the seven patterns above.
4. Pick your top 3 actions, fix them, re-baseline.
5. Repeat.

The agencies that win the enterprise client meeting are the ones that can describe their posture in this language.

What's NOT in this brief

This is patterns, not incidents. Specific named attacks change quarterly. The patterns don't. When a new named worm shows up in 2026 or 2027, it'll be a variant of one of these seven. Your defense posture covers them all.

For named incidents, the live feeds (OSV.dev, GitHub Security Advisories) are authoritative. `/security-radar` pulls from them.
